

软件工作量估算的负面结果

蒂姆·门齐斯¹·杨烨²·乔治·马修¹·巴里·
博姆³·贾鲁斯·希恩⁴

在线发布时间：2016 年 11 月 21 日
© 施普林格科学商业媒体纽约公司 2016 年

摘要 超过一半关于软件工作量估算 (SEE) 的文献聚焦于新估算方法的比较。令人惊讶的是, 尚无研究将最先进的最新方法与数十年前的方法进行比较。因此, 本文通过以下方式检验新的 SEE 方法是否比旧方法产生更优的估算结果。首先, 我们收集了从“经典 COCOMO (基于一组预先确定的属性进行参数化估算)”到“现代 (通过类比推理, 使用基于谱的聚类以及实例和特征选择, 以及《ACM 软件工程汇刊》中提出的一种近期基线方法)”等一系列工作量估算方法。其次, 我们梳理了导致后 COCOMO 估算方法发展的一系列异议。第三, 我们将这些异议中的每一个都描述为新旧估算方法之间的比较。第四,

来稿经手人: 理查德·佩奇、乔迪·卡博特和尼尔·恩斯特

蒂姆·孟席斯
tim.menzies@gmail.com

杨烨
ye.yang@stevens.edu

乔治·马修
george.meg91@gmail.com

巴里·博姆
barryboehm@gmail.com

杰鲁斯·希恩
jairus.hihn@jpl.nasa.gov

¹ 美国北卡罗来纳州罗利市北卡罗来纳州立大学计算机科学系

美国新泽西州霍博肯市史蒂文斯学院软件工程系

³ 美国加利福尼亚州洛杉矶市南加州大学计算机科学系

⁴ 美国加利福尼亚州帕萨迪纳市加州理工学院喷气推进实验室

使用四个 COCOMO 风格的数据集（来自 1991 年、2000 年、2005 年、2010 年），我们进行了这些比较实验。第五，我们使用 Scott-Knott 程序比较不同估计器的性能（该程序包括 A12 效应量以排除“微小差异”，以及 99% 置信度的自助法程序来检验处理分组在统计上是否存在差异）。本文的主要负面结果是，对于 COCOMO 数据集，我们所研究的方法没有一种比 Boehm 的原始方法表现更好。当 COCOMO 风格的属性可用时，我们强烈建议 (i) 使用该数据，以及 (ii) 使用 COCOMO 生成预测。我们这样说是因为本文的实验表明，至少对于工作量估计而言，数据的收集方式比将何种学习器应用于该数据更为重要。

关键词 工作量估算 · 构造性成本模型 (COCOMO) · 分类与回归树 (CART) · 最近邻 · 聚类 · 特征选择 · 原型生成 · 自助采样法 · 效应量 · A12

1 引言

本文探讨的是软件工作量估算方面的一个负面结果，具体而言：

对于以特定方式 (COCOMO 格式 (Boehm 等人, 2000 年)) 表示的项目数据；尽管在替代方法上已经投入了数十年的研究；

该数据的最佳预测来自 2000 年提出的一种参数化方法 (Boehm 等人, 2000 年)。

这一结论有两个需要注意的地方。首先，并非所有项目都能用 COCOMO 来表述——但当有选择时，本文的结果表明采用这种形式是有价值的。其次，我们的结论是关于单独预测方法的，这与集成方法不同 (Minku 和 Yao, 2011 年、2013 年；Kocaguneli 等人, 2012 年；Menzies 等人, 2015 年)——但如果使用集成方法，本文表明参数估计将是一个可行的集成成员。

出于务实和方法学的考虑，像上述那样报告负面结果很重要。从务实角度来讲，让行业从业者知道（有时）他们无需浪费时间费力去理解前沿技术论文是很重要的。接下来，我们将精确界定一类对前沿工作量估算技术反应不佳的项目数据。对于这类数据集，从业者可以放心，使用简单的传统方法是合理、负责且有用的。

此外，从方法论角度而言，承认错误至关重要。根据卡尔·波普尔（科学哲学领域的杰出人物（波普尔, 1963 年））的观点，“最佳”理论是那些在激烈辩论中表现最佳的理论。鉴于我们参与过一些备受瞩目的辩论（在软件分析领域（孟席斯等人, 2007 年）），我们断言此类批评非常有用，因为它们有助于研究人员：(1) 发现旧观念中的缺陷，以及 (2) 发展出更好的新观念。也就是说，发现并承认错误应被视为科学标准操作流程中的常规部分。

鉴于上述情况，令人不安的是，在软件分析领域几乎没有负面结果。偶尔出现的情况是对先前工作进行小修正的报告。考虑到软件分析的复杂性，缺乏此类失败报告非常可疑。有关此类报告的示例，请参阅（例如，如门齐斯等人 2013 年；墨菲 - 希尔等人 2012 年所做的那样）。

为什么这些报告如此罕见？原因可能有很多，在此我们推测两种可能性。首先，此类负面报告可能不被学界认为“有价值”。像本期刊这样的论坛就极为少见（这也是本期如此重要的原因）。其次，在软件分析领域，研究人员通常不会将他们的最新成果与一些所谓更简单的“稻草人”方法进行对比。科恩（1995 年）在其关于人工智能实证方法的教科书中，强烈建议进行这种“稻草人”对比，因为有时，这种对比会揭示出所谓更优越的方法实际上过于复杂。因此，将各种方法与更简单的替代方法进行比较总是有益的。

话虽如此，在某些情况下，由于没有可用的方法，此类基准测试无法进行。不过，随着某个领域逐渐走向成熟，就可以开展这些比较；例如，可参考关于缺陷预测的多项实验（斯坎涅洛等人，2013 年），或针对 Stack Overflow 帖子的标签推断（斯坦利和伯恩，2013 年）。因此，本文研究了工作量估算中一个有趣但目前尚未探索的方面。我们探究是否存在这样的数据集，在这些数据集中，非常古老的方法与其他任何方法的效果一样好。我们考虑用 COCOMO 本体表达的数据：描述软件项目的 23 个属性，以及其人员、平台和产品特性等方面。¹我们将表明，（鉴于从一个项目中收集到的数据类型多样），Boehm 2000 模型的效果与我们尝试的其他所有模型一样好（甚至更好）。因此，我们强烈建议，如果有这类数据，就应该收集起来，并使用 Boehm 2000 COCOMO 模型进行处理。

为指导我们的探索，本文提出了四个研究问题。这些问题是基于我们对 COCOMO 与其他方法优缺点的讨论经验而选定的。基于我们的经验，我们断言，以下每个问题都曾被用于推动对标准 COCOMO-II 模型的某种替代方案的开发：

研究问题 1：参数估计是否并不比仅使用代码行数度量更好？（一种常见但很少被验证的说法）

研究问题 2：参数估计是否已被更新的估计方法所取代？我们应用了我们“最佳”的学习器，以及基于案例的推理和回归树。

研究问题 3：旧的参数调整与较新的项目是否无关？COCOMO 模型是通过使用本地项目数据“调整”默认模型参数来学习的。COCOMO-II 附带了一组从 1995 年至 2000 年的特定项目集中学习到的参数。我们将这些未经修改的 COCOMO-II 调整应用于 1970 年至 2010 年的广泛项目。

研究问题 4：在某些新站点部署参数估计的成本是否高昂？我们尝试在小训练集上调整估计模型，并简化项目的规格说明。

为了探究这些问题，我们使用 COCOMO，因为其内部细节已完全公开（Boehm 等人，2000 年）。此外，我们可以获取 2000 年 COCOMO 模型的完整实现。进一步地，我们能够访问众多有趣的 COCOMO 数据集：见图 1 和图 2。除了一个例外，我们的学习实验不使用生成标准 COCOMO 的数据。这个例外是 COC81 数据，它使我们能够将新方法用于制定标准 COCOMO 的劳动密集型方法进行比较，见图 2。

¹ 关于这些属性的完整细节，见本文第 4 节。

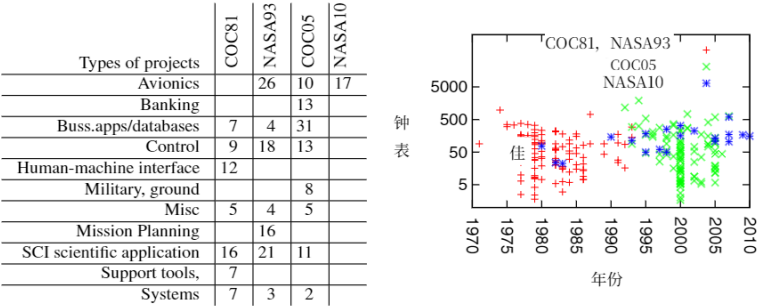


图1 本研究中学习者使用的项目。图4展示了项目属性。COC81是1981年COCOMO书籍(Boehm, 1981年)中的原始数据。这些数据来自1970年至1980年的项目。NASA93是美国国家航空航天局(NASA)在20世纪90年代初收集的关于支持国际空间站规划活动的软件的数据。我们的其他数据集是NASA10和COC05(后者属于专有数据,无法向研究界发布)。非专有数据(COC81、NASA93和NASA10)可在<http://openscience.us/repo>获取,或见图3。

利用这些数据,本文的实验得出结论,我们所有研究问题的答案几乎总是“否”。RQ1实验表明,好的估算会使用许多变量,而较差的估算则源于一些基于千行代码数(KLOC)的陈腐计算。至于其他研究问题(RQ2、RQ3、RQ4),这些结果表明,参数估算的持续使用仍然是可行的——至少对于用23个COCOMO属性表示的数据来说是这样。

如需查看我们的数据样本,请参见图3中的NASA10数据集。

2 关于工作量估算

2.1 历史

准确估算软件开发工作量至关重要。估算不足可能导致进度和预算超支,甚至项目取消(Spareref.com, 2002年)。估算过高则会延迟对其他有前景的想法的资金投入,并影响组织竞争力(科查古内利等人, 2012年)。因此,长期以来,研究人员一直在探索软件工作量估算方法,例如沃尔弗顿(1974年)、弗赖曼和帕克(1979年)、普特南(1976年)、布莱克等人(1977年)、赫德等人(1977年)、沃尔斯顿和费利克斯(1977年)、詹森(1983年)、帕克(1988年)、博姆(1981年)、沃克登和杰弗里(1999年)、谢泼德和斯科菲尔德(1997年)。

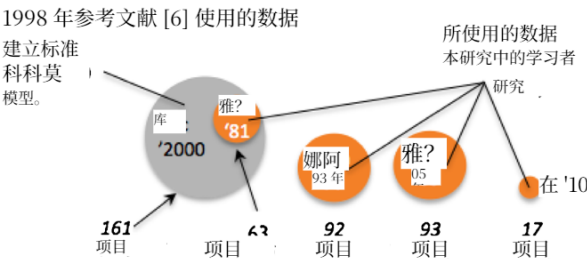


图2 本研究中的项目。COC81是COCOMO-II的一个子集。请注意,NASA'93、COC'05和NASA'10与本文中用于定义COCOMO版本的数据没有重叠。

prec	flex	resl	team	pmat	rely	cplx	data	ruse	time	stor	pvol	acap	pcap	pcon	aexp	plex	ltex	tool	seed	site	docu	kloc	months
2	2	2	3	3	4	5	4	3	5	6	4	4	4	3	4	3	3	1	3	4	4	77	1830
2	2	2	3	3	5	5	2	3	5	6	2	4	3	3	2	1	2	2	3	4	4	24	648
2	2	2	3	3	4	5	3	3	5	5	4	3	3	3	3	2	2	1	3	4	4	23	492
2	2	3	3	2	4	4	3	2	3	3	4	3	3	3	3	3	4	2	3	5	3	146	3292
2	3	3	5	3	3	4	3	2	4	4	2	5	5	4	5	1	5	3	3	6	3	113	1080
3	3	3	3	3	3	4	3	2	3	3	3	3	3	3	4	3	4	2	3	4	3	184	1043
5	3	3	3	4	4	4	3	2	3	3	2	3	3	3	5	3	4	2	3	5	3	61	336
5	3	3	4	4	4	5	3	2	3	3	2	3	3	3	5	3	4	2	3	6	3	50	637
3	3	3	2	3	4	5	3	2	3	3	3	3	3	3	4	3	4	2	3	5	3	253	2519
3	3	4	3	3	4	4	3	4	3	3	2	3	4	3	3	1	4	5	3	2	3	159	1048
3	3	3	3	3	4	5	3	2	3	3	4	4	4	5	4	4	4	2	1	5	3	324	1735
3	2	4	4	3	4	5	3	4	3	4	5	4	4	3	4	4	3	4	2	6	3	224	691
5	2	2	4	3	4	3	3	4	5	4	3	4	4	3	4	4	4	3	3	3	3	105	320
3	2	2	4	3	4	3	3	3	3	2	4	4	4	3	4	4	4	3	3	3	3	173	329
3	2	4	3	3	4	5	3	4	3	3	4	3	4	4	4	3	3	3	3	5	3	597	1705
4	2	4	3	5	4	3	2	3	3	4	4	2	2	3	3	5	5	3	3	5	3	155	789
4	3	3	3	4	4	4	3	2	3	3	3	4	4	3	5	4	4	2	3	5	3	170	552
scale factors					effort multipliers																	size	effort

图 3 NASA10 中的 17 个项目（每个项目一行）。关于第 1 行中术语（“prec”、“flex”、“resl”等）的定义，见图 4。对于不同的列，规模因子对工作量有指数级影响，而工作量乘数对工作量有线性影响。任何值为“3”的工作量乘数都是标称值；即，它将工作量乘以 1.0 的倍数。“3”以上和以下的工作量乘数都可以使项目工作量产生 0.7 到 1.74 的倍数变化。有关这些值的具体使用方法，见图 5。

约根森和谢泼德（2007 年）、门齐斯等人（2006 年），以及伯吉斯和莱夫利（2001 年）。2007 年，约根森和谢泼德报告了数百篇可追溯到 20 世纪 70 年代的关于该主题的研究论文，其中一半以上提出了一些开发新估算模型的创新方法（约根森和谢泼德，2007 年）。从那时起，又发表了许多此类论文，例如洛坎和门德斯（2006 年）、科拉扎等人（2010 年）、明库和姚（2014 年）、李和鲁赫（2007 年）、李等人（2009 年）、强（2008 年）、强等人（2008 年）、强和基奇纳姆（2008 年）、科查古内利等人（2012 年）、门齐斯和谢泼德（2012 年）、科查古内利等人（2013 年），以及科查古内利等人（2014 年）。

在 20 世纪 70 年代和 80 年代，这类研究主要集中在参数估计上，就像普特南等人所做的那样（沃尔弗顿，1974 年；弗赖曼和帕克，1979 年；布莱克等人，1977 年；赫德等人，1977 年；沃尔斯顿和费利克斯，1977 年；博姆，1981 年）。例如，博姆的构造性成本模型（COCOMO）（博姆，1981 年）假设，工作量随规模呈指数变化，如以下参数形式所示： $\text{effort} = a \times K \text{LOC}^b$ 。要在一个组织中应用这个公式，需使用本地项目数据来调整 (a, b) 参数值。如果没有本地数据，新项目可以重用先前的调整结果，并稍作修改

（门齐斯等人，2005 年）。COCOMO 是一种参数化方法；也就是说，它是一种基于模型的方法，该方法（a）假设目标模型具有特定结构，然后（b）使用基于模型的方法来填充该结构的细节（例如设置一些调整参数）。

自开展参数估计研究以来，研究人员基于回归树（谢泼德和斯科菲尔德，1997 年）、基于案例的推理（谢泼德和斯科菲尔德，1997 年）、谱聚类（门齐斯等人，2013 年）、遗传算法（科尔德罗等人，1997 年；伯吉斯和莱夫利，2001 年）等创新了其他方法。这些方法可以通过禁忌搜索（科拉扎等人，2010 年）、特征选择（陈等人，2005 年）、实例选择（科查古内利等人，2012 年）、特征合成（门齐斯和谢泼德，2012 年）、主动学习（科查古内利等人，2013 年）、迁移学习（科查古内利等人，2014 年）、时间学习（洛坎和门德斯，2009 年；明库和姚，2014 年）等“元级”技术进行增强，此外还有更多技术（图 4）。

	Definition	Low-end = {1,2}	Medium = {3,4}	High-end= {5,6}
--	------------	-----------------	----------------	-----------------

比例因子:

Flex	development flexibility	development process rigorously defined	some guidelines, which can be relaxed	only general goals defined
Pmat	process maturity	CMM level 1	CMM level 3	CMM level 5
Prec	precedentedness	we have never built this kind of software before	somewhat new	thoroughly familiar
Resl	architecture or risk resolution	few interfaces defined or few risks eliminated	most interfaces defined or most risks eliminated	all interfaces defined or all risks eliminated
Team	team cohesion	very difficult interactions	basically co-operative	seamless interactions

工作量乘数

acap	analyst capability	worst 35%	35% - 90%	best 10%
aexp	applications experience	2 months	1 year	6 years
cplx	product complexity	e.g. simple read/write statements	e.g. use of simple interface widgets	e.g. performance-critical embedded systems
data	database size (DB bytes/SLOC)	10	100	1000
docu	documentation	many life-cycle phases not documented		extensive reporting for each life-cycle phase
ltex	language and tool-set experience	2 months	1 year	6 years
pcap	programmer capability	worst 15%	55%	best 10%
pcon	personnel continuity (% turnover per year)	48%	12%	3%
plex	platform experience	2 months	1 year	6 years
pvol	platform volatility (<i>frequency of major changes</i> <i>frequency of minor changes</i>)	<i>12 months</i> <i>1 month</i>	<i>6 months</i> <i>2 weeks</i>	<i>2 weeks</i> <i>2 days</i>
rely	required reliability	errors are slight inconvenience	errors are easily recoverable	errors can risk human life
ruse	required reuse	none	multiple program	multiple product lines
sced	dictated development schedule	deadlines moved to 75% of the original estimate	no change	deadlines moved back to 160% of original estimate
site	multi-site development	some contact: phone, mail	some email	interactive multi-media
stor	required % of available RAM	N/A	50%	95%
time	required % of available CPU	N/A	50%	95%
tool	use of software tools	edit,code,debug		integrated with life cycle

人工月 建设所需人工月数 1 个人工月 = 152 小时 (包含开发和管理工时)。

图 4 COCOMO-II 属性

2.2 当前实践

玛丽·肖 (Shaw, 2001 年) 在 ICSE' 01 大会的主题演讲中指出, 科研创新成果通常需要长达十年的时间才能趋于稳定, 之后还需再过十年才能广泛普及。照此推断, 20 世纪 90 年代关于回归树的估算研究成果 (Shepperd 和 Schofield, 1997 年) 或基于案例推理的研究成果 (Shepperd 和 Schofield, 1997 年) 应该已经在商业领域得到应用。然而, 事实并非如此。

参数估计应用广泛，尤其是在航空航天工业及美国各政府机构中。例如：

- 美国国家航空航天局（NASA）会定期检查 COCOMO 中的软件估算（达布尼，2002 年）。
- 在我们与中国和美国软件行业的合作中，我们看到几乎只使用参数估算工具，比如 Price Systems (pricesystems.com) 和 Galorath (galorath.com) 提供的那些工具。
- 专业协会、手册和认证项目大多围绕参数估算方法和工具展开；例如，可参考国际成本估算与分析协会；美国国家航空航天局（NASA）成本研讨会；COCOMO 与系统 / 软件成本建模国际论坛（相关网站：<http://tiny.cc/iceaa>、<http://tiny.cc/nasa> cost、<http://tiny.cc/csse>）。

2.3 但是有人使用 COCOMO 吗？

关于工作量估算，存在两个误区：（1）没人会使用像 COCOMO 这样基于模型的估算方法；（2）基于专家经验的猜测估算总是更好。

这些说法具有误导性。如上文所述，基于模型的参数化方法在工业中广泛应用，并且得到专业协会的大力推崇。此外，虽然基于专家的估算确实是一种常见做法（Boehm，2000 年），但这并不意味着应将其推荐为进行估算的最佳或唯一方法：

- 约根森（2004 年）回顾了比较基于模型和基于专家的估计的研究，并得出结论，没有明确证据表明专家方法更优。
- 2015 年，乔根森进一步指出（乔根森，2015 年），基于模型的方法有助于了解特定估计值的不确定性；例如，通过多次运行这些模型，每次对输入数据进行微小变动。
- 瓦莱迪（2011 年）列举了可能导致专家给出糟糕专家评估的认知偏差。
- 帕索斯等人表明，许多商业软件工程师会将最初几个项目的经验推广到未来所有项目中（帕索斯等人，2011 年）。
- 约根森和格鲁施克（2009 年）记录了商业“专家”如何很少借鉴以往项目的经验教训来改进他们未来的专家估计。他们给出了一些例子，说明这种未能修正先验信念的情况如何导致了基于专家的不良估计。

许多研究得出结论，最佳估计值来自于综合多个预言模型的预测结果（科查古内利等人，2012 年；丘拉尼等人，1999 年；贝克，2007 年；瓦勒迪，2011 年）。请注意，与比较多个专家团队的估计值相比，使用专家加基于模型的方法来应用这种双重检查策略要容易得多。例如，本文研究的所有基于模型的方法都能在短短几秒钟内生成估计值。相比之下，基于专家的估计要慢几个数量级——正如瓦勒迪的 COSYSMO 专家方法所示。尽管瓦勒迪是这种方法的强烈支持者，但他也承认，“当需要大样本量时，（这）极其耗时”（瓦勒迪，2011 年）。例如，他曾招募 40 名专家参加三次专家会议，每次会议持续三个小时。假设一天工作 7.5 小时，那么这项研究耗时 $3 \times 3 \times 40 / 7.5 = 48$ 天。

COSYSMO 是一种精细的基于专家的方法。另一种更轻量级的专家方法是“计划扑克”（莫洛肯 - 普斯特沃尔德等人，2008 年），参与者在此方法中会给出

匿名对项目的完成时间进行“出价”。如果出价差异很大，那么就会详细阐述并讨论导致分歧的因素。这种出价 + 讨论的循环会一直持续，直到达成共识。

虽然计划扑克在敏捷社区中得到广泛推崇，但令人惊讶的是，评估这种方法的研究却很少（一个罕见的例外是莫洛肯 - 普斯特沃尔德等人 2008 年的研究）。此外，计划扑克用于评估 Scrum 待办事项列表中特定任务的工作量，这与对大型项目的初步估算相比，是一项不同且更简单的任务。这是一个重要问题，因为对于较大的项目，初始预算分配可能需要在存在竞争利益的团体之间进行大量的组织内部游说。对于这类需要大量估算的项目，改变初始预算分配可能具有挑战性。因此，尽可能准确地进行初始估算非常重要。

2.4 构造性成本模型 (COCOMO): 起源与发展

这些对基于专家评估的担忧可以追溯到几十年前，也是构造性成本模型 (COCOMO) 的起源。1976 年，罗伯特·瓦尔奎斯特 (TRW 公司的一位部门总经理) 告诉博姆：

在过去三周里，我不得不签署一些提案，这些提案让我们的预算超过了 5000 万美元，而原计划是 1 亿美元，但这些预算估计是提案团队中现有最优秀专家的共识。我们得做得更好。可以随时联系在以往软件成本方面有数据的专家和项目。

TRW 此前有一个模型，在 TRW 软件业务的一部分中运行良好 (沃尔弗顿, 1974 年)，但它与 TRW 业务范围内的各类嵌入式软件、指挥与控制软件以及工程和科学软件的相关性并不理想。能够接触到专家和数据是一个难得的机会，于是由雷·沃尔弗顿、库尔特·菲舍尔和 Boehm 组成的团队召开了一系列会议，并开展了专家研讨活动，以确定各种成本驱动因素的相对重要性。结合当地的专业知识和数据，再加上一些先前的研究成果，如普特南 (1976 年)、布莱克等人 (1977 年)、赫德等人 (1977 年) 以及沃尔斯顿和费利克斯 (1977 年) 的研究，还有 RCA PRICE S 模型的早期版本 (弗里曼和帕克, 1979 年)，一个名为 SCEP (软件成本估算程序) 的模型应运而生。除了一个可以解释的异常值外，对 20 个拥有可靠数据的项目的估算与实际成本的误差在 30% 以内，大多数在 15% 以内。

在从后续的 TRW 项目以及加州大学洛杉矶分校 (UCLA) 和南加州大学 (USC) 软件工程课程的约 35 个项目，连同软件成本估算的商业短期课程中收集了更多数据后，博姆 (Boehm) 得以收集到 63 个可发布的数据点，并将该模型扩展为涵盖其他软件开发模式，包括商业数据处理等其他类型的软件。由此产生的模型被称为构造性成本模型

(Constructive Cost Model)，即 COCOMO，并与相关数据一同发表在《软件工程经济学》(Boehm, 1981 年) 一书中。在 COCOMO-I 中，项目属性仅使用几个粗粒度值 (极低、低、标称、高、极高) 进行评分。这些属性是工作量乘数，当值偏离标称值时，估算值会以大于或小于 1 的某个数值变化。在 COCOMO-I 中，所有属性 (除千行代码数 (KLOC) 外) 都以线性方式影响工作量。

在发布 COCOMO-I 模型后，博姆 (Boehm) 为使用 COCOMO 模型的行业组织创建了一个联盟。该联盟收集了来自商业、航空航天、政府和非营利组织的 161 个项目的信息。基于对这些信息的分析，

对于这 161 个项目, Boehm 添加了名为规模因子的新属性, 这些属性对工作量有指数级影响 (例如, 其中一个属性是过程成熟度)。利用这些新数据, Boehm 和他的同事开发了图 5 中所示的调整参数, 将项目描述符 (极低、低等) 映射到 COCOMO-II 模型 (2000 年发布 (Boehm 等人, 2000 年)) 中使用的特定值:

$$effort = a \prod_i EM_i * KLOC^{b+0.01 \sum_j SF_j}$$

在此, EM、SF 分别是工作量乘数和规模因子, a、b 是局部校准参数 (默认值分别为 2.94 和 0.91)。此外, 工作量以“开发月”为度量单位, 1 个月相当于 152 小时的工作时间 (包括开发和管理时间)。例如, 如果 $effort = 100$, 那么根据 COCOMO 模型, 五名开发人员将在 20 个月内完成该项目。

```

= None; Coc2tunings = [
#           vlow  low  nom   high  vhigh  xhigh
# scale factors:
'Flex', 5.07, 4.05, 3.04, 2.03, 1.01, _], [
'Pmat', 7.80, 6.24, 4.68, 3.12, 1.56, _], [
'Prec', 6.20, 4.96, 3.72, 2.48, 1.24, _], [
'Resl', 7.07, 5.65, 4.24, 2.83, 1.41, _], [
'Team', 5.48, 4.38, 3.29, 2.19, 1.01, _], [
# effort multipliers:
'acap', 1.42, 1.19, 1.00, 0.85, 0.71, _], [
'aexp', 1.22, 1.10, 1.00, 0.88, 0.81, _], [
'cplx', 0.73, 0.87, 1.00, 1.17, 1.34, 1.74], [
'data', _ , 0.90, 1.00, 1.14, 1.28, _], [
'docu', 0.81, 0.91, 1.00, 1.11, 1.23, _], [
'ltex', 1.20, 1.09, 1.00, 0.91, 0.84, _], [
'pcap', 1.34, 1.15, 1.00, 0.88, 0.76, _], [
'pcon', 1.29, 1.12, 1.00, 0.90, 0.81, _], [
'plex', 1.19, 1.09, 1.00, 0.91, 0.85, _], [
'pvol', _ , 0.87, 1.00, 1.15, 1.30, _], [
'rely', 0.82, 0.92, 1.00, 1.10, 1.26, _], [
'ruse', _ , 0.95, 1.00, 1.07, 1.15, 1.24], [
'sced', 1.43, 1.14, 1.00, 1.00, 1.00, _], [
'site', 1.22, 1.09, 1.00, 0.93, 0.86, 0.80], [
'stor', _ , _ , 1.00, 1.05, 1.17, 1.46], [
'time', _ , _ , 1.00, 1.11, 1.29, 1.63], [
'tool', 1.17, 1.09, 1.00, 0.90, 0.78, _]]

def COCOMO2(project, a = 2.94, b = 0.91, # defaults
              tunes= Coc2tunings): # defaults
    sfs,ems,kloc = 0, 5, 22
    scaleFactors, effortMultipliers = 5, 17

    for i in range(scaleFactors):
        sfs += tunes[i][project[i]]

    for i in range(effortMultipliers):
        j = i + scaleFactors
        ems *= tunes[j][project[j]]

    return a * ems * project[kloc] ** (b + 0.01*sfs)

```

图 5 COCOMO-II: 一个项目的工作量估算。此处, 项目有 5 个规模因子加上 17 个工作量乘数再加上千行代码数 (KLOC)。“Xhigh”表示“极高”。除 KLOC 和工作量外, 每个属性都使用量表 $low = 1$ 、 $low = 2$ 进行评分, 最高到 $xhigh = 6$ 。请注意, 所有属性的范围从极低到极高, 因为在 Boehm 的建模工作中, 并非所有影响都涵盖整个范围。关于绿色显示的属性的解释, 见图 4。

2.5 构造性成本模型 (COCOMO) 与局部校准

COCOMO 模型是通过使用本地项目数据“调整”默认模型参数来学习的。当本地数据稀缺时，可以使用近似值，仅通过少量示例来调整模型。

例如，COCOMO 的局部校准过程，通过调整公式 (1) 中的 a 、 b 值，来调整规模因子和工作量乘数的影响，同时保持图 5 所示的调整矩阵的其他值不变。实际上，局部校准将一个 23 变量模型精简为一个双变量模型：（一个变量用于调整线性工作量乘数，另一个变量用于调整指数规模因子）。

孟席斯 (Menzies) 偏爱的局部校准程序是图 6 中的 COCONUT 程序 (2002 年首次编写，2005 年首次发表 (孟席斯等人，2005 年))。在进行若干次重复过程中，COCONUT 会通过将对 (a, b) 的一些猜测应用于某些训练数据，来评估这些猜测。如果这些猜测中有任何一个被证明是有用的 (即减少估计误差)，那么 COCONUT 会在将 (a, b) 的猜测范围缩小一定量 (比如，

```
def COCONUT(training,                # list of projects
             a=10, b=1,              # initial (a,b) guess
             deltaA = 10,            # range of "a" guesses
             deltaB = 0.5,           # range of "b" guesses
             depth = 10,             # max recursive calls
             constricting=0.66):     # next time, guess less

    if depth > 0:
        useful, a1, b1 = GUESSES(training, a, b, deltaA, deltaB)

        if useful: # only continue if something useful
            return COCONUT(training,
                            a1, b1, # our new next guess
                            deltaA * constricting,
                            deltaB * constricting,
                            depth - 1)

    return a, b

def GUESSES(training, a, b, deltaA, deltaB,
             repeats=20): # number of guesses

    useful, a1, b1, least, n = False, a, b, 10**32, 0

    while n < repeats:
        n += 1
        aGuess = a1 - deltaA + 2 * deltaA * rand()
        bGuess = b1 - deltaB + 2 * deltaB * rand()
        error = ASSESS(training, aGuess, bGuess)

        if error < least: # found a new best guess
            useful, a1, b1, least = True, aGuess, bGuess, error

    return useful, a1, b1

def ASSESS(training, aGuess, bGuess):

    error = 0.0

    for project in training: # find error on training
        predicted = COCOMO2(project, aGuess, bGuess)
        actual = effort(project)
        error += abs(predicted - actual) / actual

    return error / len(training) # mean training error
```

图 6 COCONUT 对图 5 中 COCOMO 函数的 a 、 b 进行调整

(缩小三分之二)。当出现以下情况时，COCONUT 终止：(a) 在当前递归层级未找到更好的结果，或者 (b) 经过 10 次递归调用后 —— 此时猜测范围已缩小至初始范围的 $(2/3)^{10} \approx 1\%$ 。

. 实验方法

在本节中，我们将讨论用于探究引言中所定义研究问题的方法。

3.1 实验装置的选择

“生态推断”是一种自负的观点，即“适用于全体的情况，也适用于群体中的部分个体”（波斯内特等人，2011 年；孟席斯等人，2013 年）。为避免生态推断，我们在图 7 中的装置对每个数据集分别运行

由于我们的一些方法包含随机算法（图 6 中的 COCONUT 算法），我们将实验装置重复 $N = 10$ 次（选择 10 次是因为经过实验，我们发现在 $N = 8$ 和 $N = 16$ 时结果看起来相同）需要注意的是，图 7 是一个“留一法实验”；也就是说，训练是在除一个样本之外的所有样本上进行的，然后在训练中未见过的“留出”样本上进行测试。在本研究中，训练数据和测试数据的这种分离尤为重要。如图 1 所示，我们的数据集（NASA10、COC81、NASA93 和 COC05）分别包含 17、63、92 和 93 个项目的信息。当拟合标准 COCOMO 模型的 24 个参数时（如图 4 所示），可能没有足够的信息来约束学习 —— 这意味着从理论上讲，数据几乎可以拟合任何东西（包括虚假噪声）。为了检测这种虚假模型，根据一些外部来源（如留出样本）对学习到的模型进行测试至关重要。

我们通过标准化误差 (SE) 评估性能；即

$$SE = \frac{\sum_{i=1}^n (abs(actual_i - predicted_i)) / n}{\sum_{i=1}^n (abs(actual_i - sampled_i)) / n} * 100$$

```
def RIG():
    DATA = { COC81, NASA83, COC05, NASA10 }
    for data in DATA: # e.g. data = COC81
        errors= {}
        for learner in LEARNERS: #e.g. learner=COCONUT
            for n in range(10): # ten times repeat
                for project in DATA: # e.g. one project
                    training = data - project # leave-one-out
                    model = learn(training)
                    estimate = guess(model, project)
                    actual = effort(project)
                    error = abs(actual - estimate)/actual
                    errors[learner][n] = error
            print rank(errors) # some statistical tests
```

图 7 本文使用的实验装置

该度量源自 Shepperd 和 MacDonnell 在 2012 年定义的标准化准确率 (SA) ($SE = 1 - SA$)。在公式 (2) 中, actual 代表真实值, predicted 代表预测器的估计值, sampled 是从训练数据的随机样本列表 (有放回) 中随机抽取的值。Shepperd 和 MacDonnell 还提出了另一种度量, 该度量将性能报告为某种其他简单得多的 “稻草人” 方法的比率 (他们建议使用训练数据的 $N > 100$ 个随机样本的平均工作量值)。

3.2 学习器的选择

我们的 LOC (n) “稻草人” 估计器仅使用最近项目中的代码行数。对于距离, 我们使用:

$$dist(x, y) = \sqrt{\sum_i w_i (x_i - y_i)^2} \quad (3)$$

其中 x_i , y_i 是在 $\min..max$ 范围内归一化到 0..1 的值, w_i 是加权因子 (默认为 $w_i = 1$)。在为 $n > 1$ 个邻居进行估计时, 我们通过 Walkerdén 和 Jeffery (1999) 的三角函数来合并估计值; 例如, 对于 $loc(3)$, 来自第一、第二和第三近邻的估计值分别为 a、b 和 c, 则估计值为

$$effort = (3a + 2b + 1c)/6 \quad (4)$$

我们还使用 Whigham 等人 (2015 年) 提出的自动变换线性基线模型 (ATLM) 对 COCOMO-II 和 COCONUT 模型进行基线评估。ATLM 是一种多元线性回归模型, 形式为

$$effort_i = \beta_0 + \beta_1 \cdot x_{1i} + \beta_2 \cdot x_{2i} + \dots + \beta_n \cdot x_{ni} + \epsilon_i$$

其中 $effort$ 是项目 i 的量化响应 (工作量), x_i 是描述项目的自变量。预测权重 β_i 使用最小二乘误差估计来确定。还会根据自变量 (x_i) 的性质对其进行变换。如果变量本质上是连续的, 则采用对数变换、平方根变换或不进行变换, 以使训练集中自变量的偏度最小化。如果变量是分类性质的, 则不对模型进行变换。

除了 LOC “稻草人” 和 ATLM 基线外, 我们还将 COCOMOII 和 COCONUT 与 CART 和 Knear (n) 进行比较, 因为它们在 20 世纪 90 年代已证明了自身价值 (谢泼德和斯科菲尔德, 1997 年; 沃克登和杰弗里, 1999 年)。也就是说, CART 和 Knear (n) 仍然具有现实意义: 2008 年和 2012 年 IEEE TSE 的最新研究结果仍认可将它们用于工作量估算 (德亚热等人, 2012 年; 科查古内利等人, 2012 年; 强等人, 2008 年)。此外, 根据肖提出的行业采用研究创新时间表 (引言中讨论过), CART 和 Knear (n) 现在应该已经足够成熟, 可以用于工业领域。另外, 为涵盖一些关于工作量估算的最新研究, 我们还使用了 TEAK 和 PEEKING2 (科查古内利等人, 2012 年; 帕帕克罗尼, 2013 年)。

分类与回归树 (CART, 布雷曼等人, 1984 年) 是一种迭代二分算法, 它寻找能最大程度划分数据的属性, 以使每个划分中目标变量的方差最小化。然后, 该算法对每个划分进行递归。最后, 对叶节点划分中的成本数据求平均值, 以生成估计值。Knear (n) 通过最近邻方法估计新项目的工作量 (谢泼德和斯科菲尔德, 1997 年)。与 LOC (n) 不同, Knear (n) 方法使用所有属性 (所有规模因子、工作量乘数以及代码行数) 在训练数据中找出最相近的项目。Knear (3) 使用公式 (4) 合并来自三个最近邻的工作量。Knear (n) 是一个示例, 用于说明

基于案例的推理 (CBR)，即基于案例的推理方法。1997 年，谢泼德 (Shepperd) 和斯科菲尔德 (Schofield) 首次将基于案例的推理用于工作量估算 (Shepperd 和 Schofield, 1997 年)。从那时起，它在软件工作量估算中得到了广泛应用 (奥尔等人, 2006 年; 沃克登和杰弗里, 1999 年; 柯索普和谢泼德, 2002 年; 谢泼德和斯科菲尔德, 1997 年; 门田等人, 2000 年; 李和鲁赫, 2006 年、2007 年、2008 年; 李等人, 2009 年; 强, 2008 年; 强等人, 2008 年; 强和基奇纳姆, 2008 年)。这有几个原因。首先，即使领域数据稀疏，它也能起作用 (米尔维特等人, 2005 年)。其次，与其他预测方法不同，它不对数据分布或某些潜在的参数模型做任何假设。

TEAK 基于这样一种假设构建：虚假噪声会导致记录的工作量出现较大方差 (科查古内利等人, 2012 年)。TEAK 的预处理器按如下方式去除此类高方差区域。首先，它应用贪心凝聚聚类生成一个聚类树。接下来，它考量每个子树中观察到的工作量方差，并丢弃方差最大的子树。然后对留存下来的样本进行估计。PEEKING2 (帕帕克罗尼, 2013 年) 是一种比 TEAK 激进得多的“数据修剪器”，它结合了数据约简算子、特征加权和主成分分析 (PCA)。图 8 描述了 PEEKING2。关于 TEAK 和 PEEKING2 的一个重要细节是，当它们修剪数据时，仅对训练数据进行操作。对于给定的测试集，TEAK 和 PEEKING2 总会尝试为该测试集中的所有成员生成估计值。

3.3 统计排序方法的选择

图 7 所示的实验装置最后一行对学习工作量估算器的多种方法进行了排序。在本文中，这些多种方法是在每个研究问题中探索的大小为 $ls = |I|$ 的 I 种处理的范围。例如， $RQ1$ 研究了 $ls = 4$ 方法产生的输出差异：两种 COCOMO 变体以及另外两种仅使用代码行数的方法。

本研究采用米塔斯 (Mittas) 和安杰利斯 (Angelis) 在 2013 年发表于《IEEE 软件工程汇刊》(IEEE TSE) 的论文 (Mittas 和 Angelis, 2013 年) 中推荐的斯科特 - 诺特 (Scott-Knott) 程序对各种方法进行排序。该方法根据中位数得分，对包含 l 种处理方式和 (ls) 次测量结果的列表进行排序。然后，它将 l 划分为子列表 m 和 n 。

- PEEKING2 的特征加权方案根据一个属性能够在多大程度上划分和减少工作量数据的方差，来改变公式 3 中的 w_i (方差减少得越多， w_i 得分就越高)。
- PEEKING2 的主成分分析工具使用了加速主成分分析，该方法合成了新的属性 $e_1, e_2, \dots$ ，这些属性在数据中差异最大的维度上延伸，并带有属性 d 。主成分分析将冗余变量合并为一组更少的变量 (即 $e \ll d$)，因为这些冗余在空间中变成了 (近似) 平行线。对于所有此类冗余 $j \in d$ ，我们可以忽略 j ，因为在某方面发生变化的效应在另一方面也以相同方式变化。主成分分析对于跳过 d 中的噪声变量也很有用 —— 这些变量实际上被忽略了，因为它们对数据的差异没有贡献。
- PEEKING2 的原型生成器沿着加速主成分分析 (PCA) 找到的维度对数据进行聚类。然后，每个聚类将被一个“原型”替代，该原型由该聚类中所有属性的中位数生成。原型生成是处理异常值的有效工具：大量的异常值会形成它们自己的聚类；少量的异常值会在生成中位数原型时被忽略。
- PEEKING2 通过找到测试用例最近的簇，然后在该簇中找到两个最近的邻居 (其中“近”是使用公式 3 加上特征加权计算得出)，来生成该测试用例的估计值。如果这些邻居的距离为 $n_1, n_2, n_1 < n_2$ ，且工作量为 E_1, E_2 ，那么最终估计值是偏向最近邻居的外推值：

$$n = n_1 + n_2; \text{estimate} = E_1 \frac{n_2}{n} + E_2 \frac{n_1}{n}$$

图 8 PEEKING2 内部结构 (帕帕克罗尼, 2013 年)

为了使划分前后观察到的性能差异的期望值最大化。例如，对于研究问题 1 (RQ1)，我们将根据 $ls = 4$ 方法的中位数分数对它们进行排序，然后将它们分成大小分别为 ms 、 $ns \in (1, 3), (2, 2), (3, 1)$ 的三个子列表。Scott - Knott 将按如下方式声明这些划分中的一个为“最佳”。对于大小分别为 ls 、 ms 、 ns 的列表 l 、 m 、 n ，其中 $l = m \cup n$ ，“最佳”划分使 $E(A)$ 最大化；即划分前后的期望平均值之差：

$$E(\Delta) = \frac{ms}{ls} abs(m.\mu - l.\mu)^2 + \frac{ns}{ls} abs(n.\mu - l.\mu)^2$$

然后，斯科特 - 诺特检验会检查那个“最佳”划分是否真的有用。为了进行该检验，斯科特 - 诺特检验会应用某种统计假设检验 H ，以检查 m 、 n 是否存在显著差异。如果存在显著差异，斯科特 - 诺特检验会对“最佳”划分的每一半进行递归处理。

举一个更具体的例子，考虑 $l = 5$ 处理的结果：

$$r \times 1 = [0.34, 0.49, 0.51, 0.6]$$

$$r \times 2 = [0.6, 0.7, 0.8, 0.9]$$

$$r \times 3 = [0.15, 0.25, 0.4, 0.35]$$

$$r \times 4 = [0.6, 0.7, 0.8, 0.9]$$

$$r \times 5 = [0.1, 0.2, 0.3, 0.4]$$

排序和划分后，Scott - Knott 宣称：

- 排名第一的是 $rx5$ ，中位数为 0.25
- 排名第一的是 $rx3$ ，中位数为 0.3
- 排名第二的是 $rx1$ ，中位数为 0.5
- 排名第三的是 $rx2$ ，中位数为 0.75
- 排名第三的是 $rx4$ ，中位数为 0.75

请注意，Scott-Knott 检验发现 $rx5$ 与 $rx3$ 之间差异不大。因此，尽管它们的中位数不同，但它们的排名相同。

斯科特 - 诺特检验比所有方法的全配对假设检验更好；例如，六种处理方式可以通过 $(6^2 - 6)/2 = 15$ 种方式进行比较。对每次比较进行 95% 置信度检验，其总体置信度非常低： $0.95^{15} = 46\%$ 为 46%。为避免全配对比较，斯科特 - 诺特检验仅在找到能使性能差异最大化的划分后才进行假设检验。

在本研究中，我们的假设检验 H 是 A_{12} 效应量检验和非参数自助抽样的结合；也就是说，如果自助抽样和效应量检验都表明划分在统计上显著 (99% 置信度) 且不是“小”效应 ($(A_{12} \geq 0.6)$)，我们的 Scott - Knott 方法就会对数据进行划分。

关于使用非参数自助法的理由，见埃弗龙 (Efron) 和蒂布希拉尼 (Tibshirani) (1993 年，第 220 - 223 页)。关于使用效应量检验的理由，见谢泼德 (Shepperd) 和麦克多内尔 (Macdonell) (2012 年)、坎佩内斯 (Kampenes) 等人 (2007 年) 以及科查古内利 (Kocaguneli) 等人 (2013 年)。这些研究人员警告说，即使假设检验表明两个总体“显著”不同，但如果“效应量”非常小，那么这个结果就具有误导性。因此，为了评估性能差异，我们首先必须排除小的效应。瓦尔加 (Vargha) 和德莱尼

(Delaney) 的非参数 A_{12} 效应量检验研究大小分别为 m 和 n 的两个列表 M 和 N

$$A_{12} = \left(\sum_{x \in M, y \in N} \begin{cases} 1 & x > y \\ 0.5 & x = y \end{cases} \right) / (mn)$$

该表达式用于计算一个样本中的数值大于另一个样本中数值的概率。这项测试最近得到了 Arcuri 和 Briand 在 2011 年国际软件工程大会 (ICSE' 11) 上的认可 (Arcuri 和 Briand, 2011 年)。

4 项结果

4.1 COCOMO 与仅代码行数的对比

本节探讨研究问题 1: 参数估计是否并不比使用简单的代码行数量更好?

对于参数估计方法, 有一种常见但却鲜少验证的批评观点, 认为它们并不比简单的代码行数量方法更好。如图 9 所示, 事实并非一定如此。该图是对代码行数 (1)、代码行数 (3)、COCOMOII 和 COCONUT 的比较排名。图 9 的各行按标准误差数值排序。这些行根据其排名进行划分, 显示在左列: 由于误差值更低, 更好的方法排名也更低。右侧列显示四分位间距 (第 25 到第 75 百分位数, 以水平线表示) 内的中位数误差 (以黑点表示)。

图 9 的关键特征在于, 仅使用代码行并不比参数估计更好。如果读者对这个结果感到惊讶, 那么我们要指出, 通过一些数学运算,

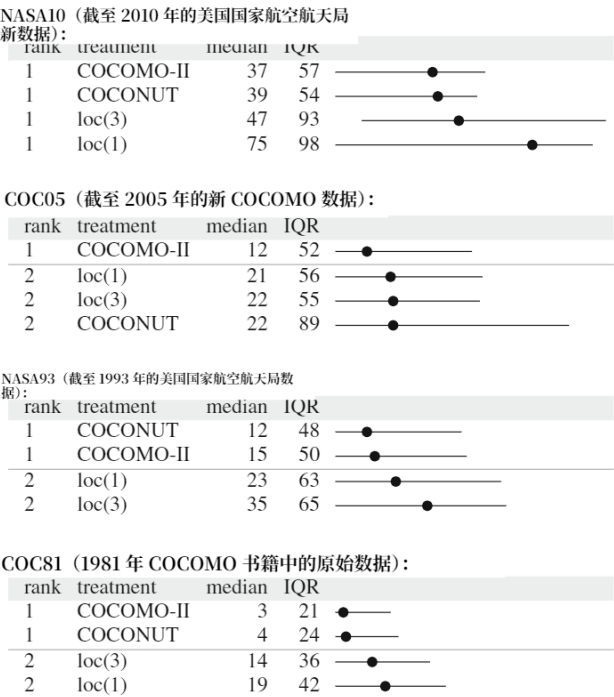


图 9 COCOMO 与仅代码行数的对比。留一法研究中观察到的标准误 (SE) 值, 重复十次。对于本图中的四个表格, 性能更好的方法在表格中位置更高。在这些表格中, 中位数 (median) 和四分位距 (IQR) 分别是第 50 百分位数和第 (75 - 25) 百分位数。四分位距范围显示在右列, 中位数处有黑点标记。水平线划分了通过我们的 Scott - Knott + 自举法 + 效应量检验得到的 “等级” (显示在左列)

可以证明，图 9 的结果并不令人惊讶。从 (1) 中可知，最小工作量受最小比例因子之和与最小工作量乘数之积的限制。对于最大工作量估计也有类似的表达式。因此，对于给定的千行代码数 (KLOC)，其取值范围由下式给出：

$$0.18 * KLOC^{0.97} \leq effort \leq 154 * KLOC^{1.23}$$

将最小值和最大值相除，得到一个表达式，该表达式表明在任何给定的千行代码数 (KLOC) 下，工作量可能会如何变化。

$$154/0.18 * KLOC^{1.23-0.97} = 856 * KLOC^{0.25}$$

公式 (6) 解释了为什么仅使用千行代码数 (KLOC) 进行估算的效果如此之差。该公式有两个组成部分：KLOC 提升至一个较小的指数 (0.25)，以及一个表示所有其他 COCOMO 变量影响的常数。第二个组成部分的 856 这一较大数值表明，KLOC 之外的许多因素都会影响工作量。因此，仅使用 KLOC 是一种糟糕的工作量估算方法也就不足为奇了。

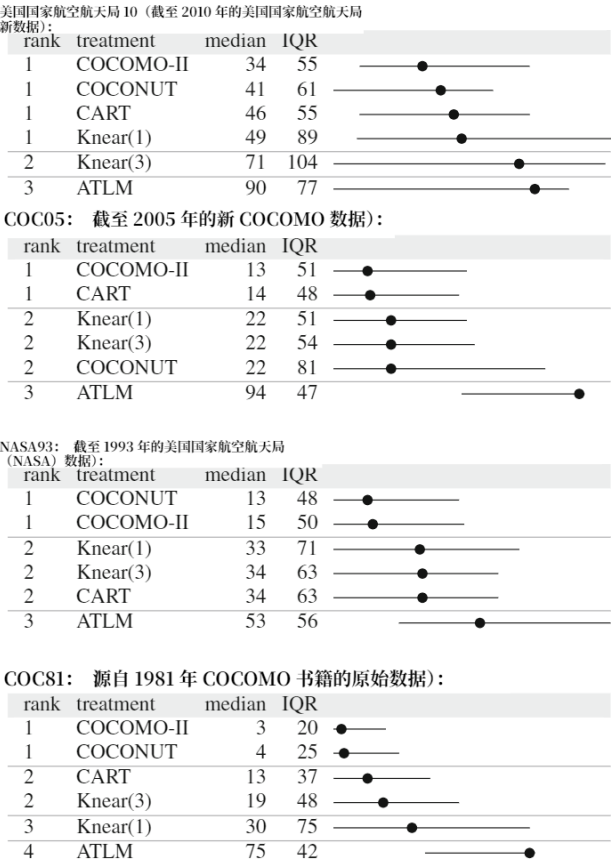


图 10 COCOMO 与标准方法对比。显示方式同图 9

4.2 COCOMO 与其他方法对比

本节探讨研究问题 2：参数估计是否已被更新的估计方法所取代？以及研究问题 3：旧的参数调整与最新项目是否无关？

图 10 将 COCOMO 和 COCONUT 与 20 世纪 90 年代的标准工作量估算方法（CART 和 Knear (n)）以及 ATLM（Whigham 等人在 2015 年提出的基线工作量估算方法 (2015 年)（其作者定义的一种方法，旨在更好地定义工作量估算实验，或许也是为了鼓励这类研究具有更高的可重复性））进行了比较。在该比较中，COCOMO-II 的误差排名并不比其他任何方法差（有时 COCONUT 的中位标准误差略低，但差异很小： $\leq 2\%$ ）。

图 11 将 COCOMO 和 COCONUT 与更新的工作量估算方法（TEAK 和 PEEKING2）进行了比较。同样，没有任何方法的排名优于 COCOMO-II 或 COCONUT。

从这些结果来看，我们建议工作量估算研究人员在将新方法 with 旧方法进行对比时应谨慎设定基准。

至于 COCONUT，该方法的排名常常与 COCOMO-II 相当。在几种情况下，COCOMO-II 和 COCONUT 分别位列第一和第二，且它们得分的中位数差异非常小。

从这些数据中，我们得出结论，参数估计已被诸如 CART、Knear (n)、TEAK 或 PEEKING2 等最新创新完全取代这一说法并不总是成立。此外，2000 年的 COCOMO-II 调整不仅对 2000 年之前的项目有用。

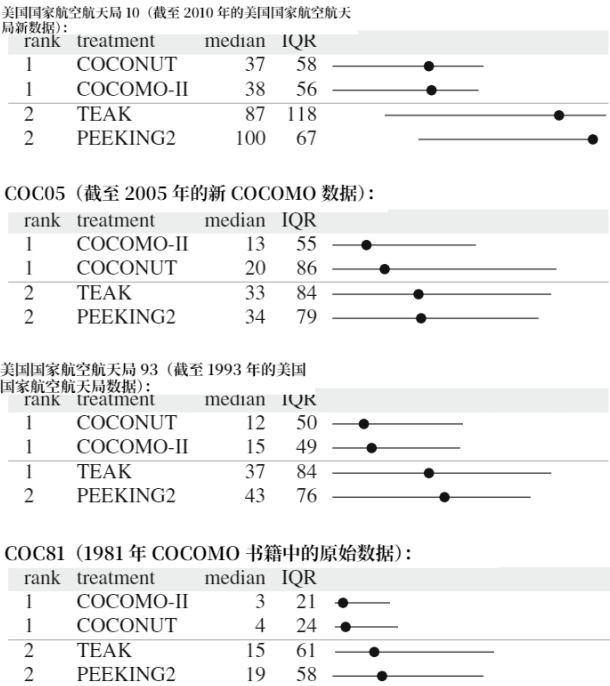


图 11 COCOMO 与更新方法的对比。按照图 9 的方式展示

(COC81 的全部内容, 以及 NASA93 的部分内容), 还适用于在这些调整之后长达十年内完成的项目 (NASA10)。

4.3 COCOMO 与简化版 COCOMO 对比

本节探讨研究问题 4: 在某些新站点部署参数估计的成本是否高昂? 为此, 我们评估对 COCOMO-II 进行某些简化所带来的影响。

4.3.1 范围缩减

在新组织中部署 COCOMO 的成本在于, 需要投入培训精力, 以便不同分析师能给出一致的项目评级。如果我们能简化目前的六级评分标准 (极低、低、一般、高、极高和极高), 那么对项目的分歧就会减少。因此, 我们尝试将六级评分标准简化为三级:

- 标称值: 与之前相同;
- 高于: 任何高于标称值的情况;
- 低于: 任何低于标称值的情况。

要做到这一点, 需更改图 5 的调谐表。对于每一行, 将标称值以下的所有值替换为其平均值 (标称值以上的值同理)。例如, 以下是缩减为低于、标称、高于之前和之后的时间调谐值:

range	vlow	low	<i>nominal</i>	high	vhigh	xhigh
before	1.22	1.09	1.00	0.93	0.86	0.80
reduced	1.15	1.15	1.00	0.863	0.863	0.863
	<i>below</i>			<i>above</i>		

4.3.2 行简化

只有在收集了数百个新示例后, 才会对新的 COCOMO 模型进行调整。如果没有这个必要, 我们有望看到多种 COCOMO 模型, 每种模型都针对不同的特定 (且规模较小) 项目样本进行调整。因此, 我们探索在非常小的数据集上对 COCOMO 进行调整。为了实现行缩减, 对训练数据进行了随机打乱, 并对所有行或仅前四行或八行进行训练 (分别用 *r4* *r8* 表示)。请注意, 鉴于使用 *r8* 获得了积极结果, 我们没有探索更大的训练集。

4.3.3 列简化

先前的结果表明, 行约简应与列约简同时进行。Chen 等人 (2005 年) 的一项研究将列约简 (剔除噪声或相关属性) 与行约简相结合。他们的结果非常明确: 随着行数的减少, 使用更少的列能得到更好的估计。Miller (2002 年) 解释了原因: 通过最小化最小二乘误差学习得到的线性模型的方差会随着模型中列数的减少而降低。也就是说, 随着列数的减少, 预测可靠性可以提高 (注意: 如果去除过多, 就没有用于预测的信息了)。

因此，本实验根据属性对特定努力值的筛选效果，对训练集中的属性进行排序。设 $x \in a_i$ 表示属性 a_i 的唯一值列表。此外，设训练数据中有 N 行；设 $r(x)$ 表示包含 x 的 n 行；设 $v(r(x))$ 为这些行中努力值的方差。“好”属性的值对特定努力的筛选效果最佳；即这些属性使 $E(\sigma, a_i) = \sum_{x \in a_i} (n/N * v(r(x)))$ 最小化。

本实验根据 $E(\sigma, a_i)$ 对所有训练数据属性进行排序，然后保留列中下半部分或一半或 *all* 的数据（分别表示为 *c0.25* 或 *c0.5* 或 *c1*）。请注意，由于图 9 的结果，代码行数（LOC）被排除在列缩减之外。

4.3.4 结果

图 12 比较了使用 *all* 或一些缩减的范围、行和列集合时得到的结果。请注意我们的命名法：COCONUT:c0.5,r8 结果是在对八个随机选择的训练示例进行训练后得到的，这些示例被缩减为低于、标称、高于，同时忽略 50% 的列。

图 12 表明，仅从 4 到 8 个项目学习中学习 COCOMO 模型是可行的。除了 COC81 的结果外，大多数 *r8* 结果都名列前茅（但即使在这种情况下，排名靠前的 *r8* 结果与标准 COCOMO 之间的绝对差异也非常小：仅为 2%）。

总体而言，图 12 表明，与 COCOMO-II 相关的建模工作可以减少。因此，在一些新的项目中部署参数估算并不一定需要高昂成本。项目属性无需详细说明：一个简单的三点量表就足够了：低于、正常、高于。至于建模需要多少数据，仅仅四到八个项目就足以进行校准。因此，应该有可能仅使用一个三点量表，快速为各种专业子群体构建许多类似 COCOMO 的模型。

· 对有效性的 5 种威胁

在我们如何选择项目（数据集）用于实验方面，出现了有效性问题。虽然我们使用了团队之间能够共享的所有数据集，但尚不清楚我们的结果是否能推广到其他尚未研究的数据集。另一方面，就参数估计文献而言，这是迄今已发表的最为广泛和详尽的研究之一。

为了提高外部有效性，本研究中使用的所有数据均可在 PROMISE 代码库中在线获取。此外，我们采用留一法实验装置，且四个数据集中有三个（NASA93、COC81、NASA10）可公开获取，这意味着其他研究人员能够完全重现我们在本研究中使用的代码和实验装置。

本研究中偏差的一个来源是用于缺陷预测研究的学习者。数据挖掘是一个庞大且活跃的领域，任何单一研究都只能使用已知数据挖掘算法的一个小子集。软件工程数据挖掘中的任何案例研究都只能探索一小部分由研究人员的偏见所选择的选项。研究人员所能期望做到的最好情况是说明他们的偏见，并尝试对其进行弥补。因此：

- 本文作者的偏好使我们选择了参数化建模方法（COCOMO）作为主要建模方法。

- 然后，我们有意识地决定扭转这些偏差，探索非参数方法（PEEKING2 和 TEAK）以及决策树方法（CART）。

6 结论

在过去的几十年里，有一系列创新方法被应用于工作量估算。本文将其中一些方法的样本与一种已存在数十年的参数估计方法进行了比较。

基于该研究，我们得出了一个负面结果，即一种沿用数十年的工作量估算方法与最新方法表现相当，甚至更优：

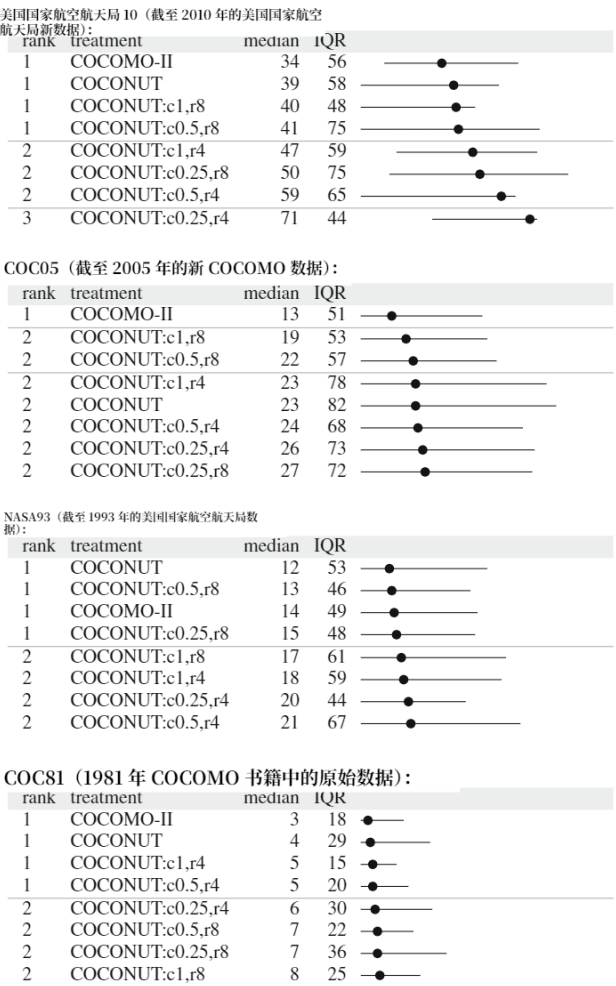


图 12 COCOMO 与简化版 COCOMO 对比。按图 9 的方式呈现

- 研究问题 1: 仅使用代码行数 (LOC) 进行估算, 远比基于多个属性的参数化估算差得多 (见 4.1 节);
- 研究问题 2: 在工作量估算方面的新创新并未取代参数估算 (见 4.2 节);
- 研究问题 3: 旧的参数调整并未过时 (见 4.2 节);
- 研究问题 4: 是否可以通过一定范围的行列剪枝简化参数估计, 以降低在新站点部署这些方法的成本 (见 4.3 节);

因此, 我们得出结论, 在 2016 年, 首先尝试参数估计仍然是一种有效且值得推荐的做法。在这些实验中, 4 到 8 个项目就足以学习到良好的预测指标 (并且我们正在探索进一步减少项目数量的方法)。这是一个重要的结果, 因为鉴于软件工程变化的快速步伐, 各组织不太可能有数十个先前的相关项目可供借鉴。

我们在这里得出的结论是, 选择收集哪些数据可能比将何种学习算法应用于这些数据更为重要。当然, 并非所有项目都能用 COCOMO 模型来描述。但如果有选择的话, 我们建议收集如图 5 所示的数据, 然后使用 COCOMO-II 对这些数据进行处理。

7 未来工作

本文的负面结果让我们对一些较新的 (且理应更好的) 工作量估算创新技术产生了质疑。软件工程 (SE) 项目数据独特且高度可变的特点, 极大地限制了单纯应用某些全新算法所获得的结果。或许未来的一个研究方向是, 探究创新的新技术如何扩展 (而非取代) 现有的、成功的估算方法。

在认可了诸如 COCOMO 等参数化方法的使用后, 有必要讨论一下该方法新版本的当前计划。软件行业最近的变化表明, 是时候修订 COCOMO-II 了。敏捷方法、Web 服务、云服务、多核芯片上的并行软件、现场可编程门阵列 (FPGA) 软件、应用程序、小部件以及以网络为中心的系统之系统 (NCSOS) 的兴起, 促使 COCOMO II 的开发者和用户开始着手对已有 14 年历史的 COCOMO II 进行升级。目前关于可能推出的 COCOMO III 的讨论, 引发了对 1981 年旧版 COCOMO 开发模式的重新审视, 因为不同的开发现象似乎会影响 Web 服务、商业数据处理、实时嵌入式软件、指挥与控制以及工程和科学应用的成本和进度。

此外, 在校准 COCOMO II 模型以及开发 COCOMO III 的过程中, 我们还注意到, 在协同良好、具有领域经验的快速开发组织中, 时间竞争型敏捷项目能够显著减少工作量并加快速度 (英戈尔德等人, 2013 年)。最后, 新兴的基于社区的软件开发, 即软件众包 (李等人, 2013 年), 对传统软件估算法则的基本假设提出了挑战。外部劳动力的获取和竞争因素正成为关键的开发影响因素, 需要进一步研究。目前正在努力对这些模型进行特征描述, 并收集数据以校准应对这些模型的模型。非常欢迎各界人士为模型的定义和校准工作贡献力量。

致谢 本文所述研究部分是在加利福尼亚理工学院喷气推进实验室, 根据与美国国家航空航天局签订的合同开展的。

行政管理。本文中以商品名、商标、制造商或其他方式提及任何特定商业产品、工艺或服务，并不构成或暗示其得到美国政府的认可。

参考文献

- 阿尔库里 A, 布赖恩特 L (2011 年)《在软件工程中使用统计检验评估随机算法的实用指南》。载于:《2011 年国际软件工程大会论文集》, 第 1-10 页
- 奥尔 M、特伦多维茨 A、格拉泽 B、豪恩施密德 E、斯特凡 B (2006 年)。基于类比的成本估算中的最优项目特征权重:改进与局限。《电气与电子工程师协会软件工程汇刊》第 32 卷: 83-92 页
- 贝克 D (2007 年) 一种基于专家和模型的混合作量估算方法。硕士学位论文, 西弗吉尼亚大学莱恩计算机科学与电气工程系, 可从 <https://eidr.wvu.edu/etd/documentdata.eTD?documentid=5443> 获取
- 布莱克 R、柯诺 R、卡茨 R、布雷 M (1977 年)。《Bcs 软件生产数据》, 最终技术报告 radc-tr-77116。波音计算机服务公司技术报告
- 博姆, B (1981 年)。《软件工程经济学》。普伦蒂斯·霍尔出版社, 恩格尔伍德克利夫斯
- 博姆·B (2000 年)《安全、简易的软件成本分析》。《IEEE 软件》: 第 14-17 页
- Boehm B, Horowitz E, Madachy R, Reifer D, Bradford KC, Steece B, Winsor Brown A, Chulani S, Abts C (2000 年)。《使用 Cocomo II 进行软件成本估算》。普伦蒂斯·霍尔出版社, 恩格尔伍德克利夫斯
- 布赖曼、弗里德曼、奥尔申、斯通 (1984 年)《分类与回归树》
- 伯吉斯首席大法官、勒夫利·马丁 (2001 年)。遗传编程能否改善软件工作量估算? 一项比较评估。《信息与软件技术》43 (14):863-873
- 陈 Z、博姆 B、孟席斯 T、波特 D (2005 年)。为软件成本建模寻找合适的数据。《IEEE 软件》第 22 卷: 第 38-46 页
- 陈 Z、门齐斯 T、波特 D (2005 年)。特征子集选择可改善软件成本估算。见: PROMISE' 05。可从 <http://menzies.us/pdf/05/fsscocomo.pdf> 获取
- 楚拉尼 S、博姆 B、斯蒂斯 B (1999 年) 实证软件工程成本模型的贝叶斯分析。《IEEE 软件工程汇刊》第 25 卷第 4 期
- 科恩·P·R (1995 年),《人工智能的实证方法》, 麻省理工学院出版社, 剑桥市
- 科拉扎 A、迪·马蒂诺 S、费鲁奇 F、格拉维诺 C、萨罗 F、门德斯 E (2010 年) 禁忌搜索在配置支持向量回归进行工作量估算方面的效果如何? 载于: 第六届软件工程预测模型国际会议论文集, PROMISE' 10, 第 4:1-4:10 页
- 科德罗 R、科斯塔马尼亚 M、帕谢塔 E (1997 年)。一种用于校准类 COCOMO 模型的遗传算法方法。载于: 第 12 届 COCOMO 论坛
- 达布尼·JB (2002 年)。独立验证与确认的投资回报率。美国国家航空航天局资助的研究。研究结果可从 <http://sarpresults.ivv.nasa.gov/ViewResearch/24.jsp> 获取。
- 德亚热格 K、韦尔贝克 W、马滕斯 D、贝森斯 B (2012 年) 用于软件工作量估算的数据挖掘技术: 一项对比研究。《电气与电子工程师协会软件工程汇刊》第 38 卷: 第 375-397 页
- 埃弗龙 B、蒂布希拉尼 RJ (1993 年)。《自助法导论》。《统计学与应用概率专论》。查普曼与霍尔出版社, 伦敦
- 弗赖曼 F、帕克 R (1979 年) 普赖斯软件模型 - 版本 3: 概述。见: 电气与电子工程师协会 - 纽约州电气与电子工程师协会分会定量软件模型研讨会论文集, 电气与电子工程师协会目录编号 TH 0067-9, 第 32-41 页
- 赫德 J、波斯拉克 J、拉塞尔 W、斯图尔特 J (1977 年)。《软件成本估算研究 —— 研究结果》, 最终技术报告, RADC-TR-77-220。技术报告, 多蒂联合公司
- 英戈尔德 D、博姆 B、库尔曼诺琼 S (2013 年)。一种估算敏捷项目流程和进度加速的模型。见:《2013 年国际软件与系统过程会议论文集》, 第 29-35 页
- 詹森·R (1983 年)。一种改进的宏观层面软件开发资源估算模型, 第 88-92 页
- 约根森 M (2015 年)《世界是扭曲的: 软件开发项目中的无知、使用、误用、误解以及如何改进不确定性分析》, 2015 年 CREST 研讨会。 <http://goo.gl/0wFHLZ>
- 约根森 M、格鲁施克 TM (2009 年) 经验教训总结会议对工作量估算和不确定性评估的影响。《IEEE 软件工程汇刊》35 卷第 3 期: 368-383 页
- 约根森 M、谢泼德 M (2007 年)。软件开发成本估算研究的系统综述。获取网址: <http://www.simula.no/departments/engineering/publications/jorgensen.2005.12>
- 约根森 M (2004 年)。软件开发工作量专家评估研究综述。《系统与软件杂志》70 (1-2):37-60
- 李 M、毛 K、杨 Y、哈曼 M (2013 年)。基于众包的软件开发任务定价。见: 国际软件工程大会 (ICSE), 新想法与新成果, 美国加利福尼亚州旧金山, 第 1205-1208 页

- 卡多达 G、卡特赖特 M、陈 L、谢泼德 M (2000 年) 运用基于案例推理预测软件项目工作量的经验
- 坎佩内斯·维格迪斯比、迪布^a·T、汉内·J·E、舍贝格·D·I·K (2007 年)。软件工程实验中效应量的系统评价。《信息与软件技术》49 (11-12):1073-1086
- 姜 JW (2008 年) 针对软件成本估算的类比 - x 实证评估。见:《ESEM'08: 实证软件工程与测量国际研讨会论文集》。美国计算机协会, 美国纽约州纽约市, 第 294-296 页
- 姜俊伟、基奇纳姆 B (2008 年), 使用类比 - x 进行软件成本估算的实验。见:《2008 年澳大利亚软件工程大会论文集》, 美国电气和电子工程师协会计算机协会, 美国华盛顿特区, 第 229-238 页
- 姜 JW、基奇纳姆 BA、杰弗里 DR (2008 年) Analogy-x: 为基于类比的软件成本估算提供统计推断。《IEEE 软件工程汇刊》34 (4):471-484
- 柯索普 C、谢泼德 M (2002 年) 利用少量训练集进行推理。《电气与电子工程师协会学报》: 第 149 页
- 科查古内利 E、门齐斯 T、贝内尔 A、姜 J (2012 年) 利用基于类比的工作量估算的基本假设。《IEEE 软件工程汇刊》28 卷: 425-438 页。可从 <http://menzies.us/pdf/11teak.pdf> 获取
- 科查古内利·E、门齐斯·T、姜·J·W (2012 年)《论集成工作量估计的价值》。《IEEE 软件工程汇刊》第 38 卷第 6 期: 1403-1416 页
- 科查古内利 E、门齐斯 T、姜 J、乔克 D、马达奇 R (2013 年)《主动学习与工作量估算: 寻找软件工作量估算数据的关键内容》。《IEEE 软件工程汇刊》第 39 卷第 8 期: 1040-1053 页
- 科查古内利 E、门齐斯 T、门德斯 E (2014 年)。工作量估算中的迁移学习。《实证软件工程》: 1-31
- 科查古内利 E、齐默尔曼 T、伯德 C、纳加潘 N、门齐斯 T (2013 年) 分布式开发有害吗? 发表于:《国际软件工程大会》, 第 882-890 页
- 李荆州、鲁赫·根瑟 (2006 年)。基于类比的工作量估算属性加权启发式方法的比较研究。载于《实证软件工程国际研讨会论文集》, 第 74 页
- 李 J、鲁赫 G (2007 年) 通过类比进行软件工作量估算的决策支持分析。见: PROMISE'07: 第三届软件工程预测模型国际研讨会论文集, 第 6 页
- 李 J、鲁赫 G (2008 年) 基于类比的软件工作量估计方法 aqua + 的属性加权启发式分析。《实证软件工程》13 卷: 63-96 页
- 李 Y、谢 M、吴 T (2009 年) 对基于类比的软件成本估算的非线性调整研究。《实证软件工程》: 603-643 页
- 洛坎 C、门德斯 E (2006 年) 使用国际软件基准组织 (ISBSG) 数据库的跨公司和单公司工作量模型: 一项进一步的重复研究。载于: ACM-IEEE 实证软件工程国际研讨会, 11 月 21-22 日, 里约热内卢
- 洛坎 C、门德斯 E (2009 年) 将移动窗口应用于软件工作量估算。载于: 2009 年第三届实证软件工程与测量国际研讨会。《2009 年实证软件工程与测量》, 第 111-122 页
- 门齐斯 T、布彻 A、科克 DR、马库斯 A、莱曼 L、舒尔 F、图尔汗 B、齐默尔曼 T (2013 年)。缺陷预测和工作量估计的局部与全局经验教训。《电气与电子工程师协会软件工程汇刊》第 39 卷第 6 期: 822-834 页。可从 <http://menzies.us/pdf/12localb.pdf> 获取
- 门齐斯 T、陈 Z、希恩 J、卢姆 K (2006 年)。选择工作量估算的最佳实践。《IEEE 软件工程汇刊》。可从 <http://menzies.us/pdf/06coseekmo.pdf> 获取。
- 门齐斯 T、德赫提亚尔 A、迪斯泰法诺 J、格林沃尔德 J (2007 年) 精确性问题。《IEEE 软件工程汇刊》。 <http://menzies.us/pdf/07precision.pdf>
- 门齐斯 T、科查古内利 E、明库 L、彼得斯 F、图尔汗 B (2015 年) 第 20 章 —— 学习机集成。载于《软件工程中的数据与模型共享》, 第 239-265 页
- 门齐斯 T、彼得斯 F、马库斯 A (2013 年)《哎呀…… (跨公司学习优化” 勘误报告)》。载于:《2013 年软件工程研究国际会议论文集》。 <http://www.slideshare.net/timmmenzies/msr13-mistake>
- 门齐斯 T、波特 D、陈 Z、希恩 J、斯图克斯 S (2005 年)。校准软件工作量模型的验证方法。载于《ICSE 会议论文集》。可从 <http://menzies.us/pdf/04coconut.pdf> 获取
- 门齐斯 T、谢泼德 M (2012 年)。软件工程预测中可重复结果特刊。《实证软件工程》17 (1-2): 1-17
- 米勒·A (2002 年)《回归中的子集选择》, 第 2 版。查普曼与霍尔出版社, 伦敦
- 明库·L·L、姚欣 (2011 年)《对于软件工作量估算的学习机集成的原则性评估》, 第 106 卷
- 明库·L·L、姚·X (2013 年)《集成与局部性: 关于改进软件工作量估算的见解》。《信息与软件技术》第 55 卷: 1512-1528 页
- 明库·LL、姚 X (2014 年) 如何在软件工作量估算中最佳利用跨公司数据? 载于:《2014 年国际软件工程大会论文集》, 第 446-456 页
- 米塔斯 N、安杰利斯 L (2013 年) 通过多重比较算法对软件成本估算模型进行排序和聚类。《IEEE 软件工程汇刊》39 卷 (第 4 期): 537-551 页

莫洛肯 - 普斯特沃尔德 K, 豪根 NC, 贝内斯塔德 HC (2008 年) 在软件项目中使用计划扑克法整合专家评估。《系统与软件杂志》81 卷: 2106 - 2117 页

墨菲 - 希尔 E、帕宁 C、布莱克 AP (2012 年)。我们如何重构以及如何知晓重构效果。《IEEE 软件工程汇刊》第 38 卷第 1 期: 第 5 - 18 页

迈尔特韦特一世、斯滕斯鲁德、谢泼德 (2005 年)。软件预测模型比较研究中的可靠性与有效性。《IEEE 软件工程汇刊》, 第 31 卷第 5 期: 380 - 391 页

帕帕克罗尼五世 (2013 年)。数据雕刻: 识别并去除数据中的无关内容。硕士学位论文, 西弗吉尼亚大学莱恩计算机科学与电气工程系

帕克 R (1988 年) 价格软件成本模型的核心方程。见: 第四届 COCOMO 用户小组会议

帕索斯 C、布劳恩 AP、克鲁泽斯 DS、门东萨 M (2011 年)。分析信念对软件项目实践的影响。载于:《2011 年软件工程评估与度量国际研讨会论文集》

波普尔·K·R (1963 年)《猜想与反驳》。劳特利奇与基根·保罗出版社

波斯内特 D、菲尔科夫 V、德万布 P (2011 年), 实证软件工程中的生态推理。见:《2011 年软件工程国际会议论文集》

普特南 L (1976 年)《一种软件开发宏观估算方法》, 第 38 - 43 页

斯坎涅洛 G、格拉维诺 C、马库斯 A、门齐斯 T (2013 年) 使用软件聚类进行类级故障预测。见:《2013 年 IEEE/ACM 第 28 届自动化软件工程国际会议论文集》(ASE)。电气与电子工程师协会, 第 640 - 645 页

肖 M (2001 年)。软件架构研究的成熟。载于《第 23 届软件工程国际会议论文集》, ICSE '01, 第 656 卷。美国电气和电子工程师协会计算机协会, 美国华盛顿特区

谢泼德 M、斯科菲尔德 C (1997 年)。利用类比法估算软件项目工作量。《IEEE 软件工程汇刊》第 23 卷第 12 期。可从<http://www.utdallas.edu/~rbanker/SE.XII.pdf> 获取。

谢泼德·M·J、麦克多内尔·S·G (2012 年)。软件项目估算中预测系统的评估。《信息与软件技术》, 54 卷第 8 期: 820 - 827 页

太空参考网 (2002 年): 美国国家航空航天局将关闭检测与发射控制系统。
<http://www.spaceref.com/news/viewnews.html?id=475>

斯坦利·C、伯恩医学博士 (2013 年) 预测 Stack Overflow 帖子的标签。见:《ICCM 会议论文集》, 2013 年卷

瓦莱迪 R (2011 年) 通过宽带德尔菲法实现专家意见的趋同: 在成本估算模型中的应用。载于: 国际系统工程协会 (Incose) 国际研讨会, 美国丹佛。获取网址: <http://goo.gl/Zo9HT>

沃克登·菲奥娜、杰弗里·R (1999 年)。基于类比的软件工作量估算实证研究。《实证软件工程》4 (2):135 - 158

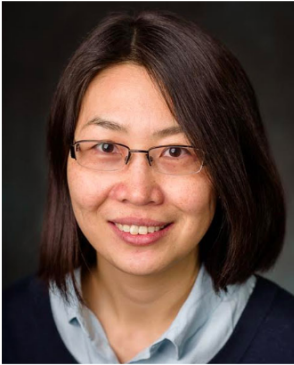
沃尔斯顿 C, 费利克斯 C (1977 年)。一种编程测量与估算方法。《IBM 系统杂志》16 (1):54 - 77

惠格姆 (Whigham) PA、欧文 (Owen) CA、麦克多内尔 (Macdonell) SG (2015 年)。一种用于软件工作量估算的基线模型。《美国计算机协会软件工程方法学汇刊》24 (3):20:1-20:11

沃尔弗顿 R (1974 年)《大规模软件开发的成本》。《电气与电子工程师协会计算机汇刊》: 第 615 - 636 页



蒂姆·门齐斯博士 (1995 年于新南威尔士大学获得博士学位) 是北卡罗来纳州立大学计算机科学系的正教授, 他在校教授软件工程、自动化软件工程以及软件科学基础等课程。他发表了 250 余篇有同行评审的论文, 并编辑了三本近期的书籍, 总结了软件分析领域的最新技术。他曾担任美国国家科学基金会 (NSF)、美国国家司法研究所 (NIJ)、美国国防部 (DoD)、美国国家航空航天局 (NASA)、美国农业部 (USDA) 等项目的首席研究员, 同时也与私营企业开展联合研究。他是《IEEE 软件工程汇刊》《实证软件工程》《自动化软件工程杂志》《大数据杂志》《信息与软件技术》以及《软件质量杂志》的副主编。更多详细信息, 请访问他的个人主页<http://menzies.us>。



叶阳博士是系统与企业学院的副教授。她的研究领域为实证软件工程，包括软件成本估算、缺陷预测和软件过程建模方法。



乔治·马修先生是北卡罗来纳州立大学（NCSSU）计算机科学专业的博士研究生。他于 2016 年在北卡罗来纳州立大学获得计算机科学硕士学位，并于 2014 年在印度阿姆里塔大学获得电子与仪器工程专业的技术学士学位。他是北卡罗来纳州立大学 RAISE 研究小组的成员。他的研究兴趣在于软件工作量估算、优化需求工程模型、分布式多目标优化以及图数据上的主题建模。



巴里·博姆博士是南加州大学杰出教授，同时担任南加州大学计算机科学、工业与系统工程以及航天学系的 TRW 教授。他还是美国国防部 - 史蒂文斯 - 南加州大学系统工程研究中心的首席科学家，以及南加州大学系统与软件工程中心的创始主任。1989 - 1992 年，他担任美国国防高级研究计划局信息科学与技术办公室 (DARPA - ISTO) 主任；1973 - 1989 年，就职于 TRW 公司；1959 - 1973 年，任职于兰德公司；1955 - 1959 年，在通用动力公司工作。他的贡献包括 COCOMO 系列成本模型、螺旋系列过程模型，以及用于创建和演进成功系统的 W 理论（双赢）方法。他是计算机领域 (ACM)、航空航天领域 (AIAA)、电子领域 (IEEE)、系统工程领域 (INCOSE) 和精益方法领域 (LSS) 主要专业学会的会士，并且是美国国家工程院院士。



贾伊鲁斯·希恩博士（马里兰大学博士）是喷气推进实验室系统工程团队的主要成员。他一直积极推动在 NASA 范围内使用定量管理和规划方法，特别是在成本估算领域。他是喷气推进实验室并行工程团队“X 团队”的风险与项目管理负责人。他发表了 60 多篇论文，包括近期发表在《IEEE 软件工程汇刊》《软件工程自动化》以及《系统与软件工程创新》上的文章。贾伊鲁斯荣获国际软件生产率组织年度参数分析师奖以及南加州大学软件工程学院终身成就奖。